SmartGranary Project Report

Project: SmartGranary

Report date: October 31, 2026

Project type: Frontend MVP / interactive prototype

1. Executive summary

SmartGranary is a browser-based agricultural inventory and granary-storage decision-support dashboard. It records crop lots, calculates storage age, assigns rule-based risk categories, displays simulated storage conditions, and surfaces inspection or selling-priority advisories.

The project is a functional user-interface prototype, not a production-connected inventory platform. It runs with seeded demonstration data and in-memory React state. It has no backend, persistent database, user authentication, live sensor feed, or market-price integration.

2. Goals and intended users

The prototype is aimed at farm managers and cooperative members who need a quick view of:

- The crops and quantities currently represented in storage.
- How long each crop lot has been stored.
- Which lots should be reviewed first under configured age rules.
- Whether simulated humidity has crossed an inspection threshold.
- A summarized inventory report suitable for printing or CSV export.

The advisory system is transparent and rule-based. It does not forecast commodity prices, guarantee profit, or replace agronomic expertise.

3. Application functionality

The application has five main dashboard sections:

Overview

Shows inventory KPIs, an inventory aging chart, the crop table, storage-condition controls, a ranked selling-priority list, and current recommendations.

Inventory

Displays crop lots with their quantity, harvest date, calculated age, location, moisture (if supplied), risk status, and advisory. Users can add, edit, or delete lots, search by crop name or location, filter by status, and export the current inventory to CSV.

Storage

Shows simulated temperature and humidity, a humidity warning, quick simulation controls, and a visualization of four named storage bays. The bay layout and ventilation descriptions are demo-oriented rather than a live facility map.

Recommendations

Lists generated advisory notices and shows a crop-by-crop rationale, including age, priority, and suggested action. Users can open the threshold configuration.

Reports

Summarizes total stock, capacity utilization, and quantity by age category. It includes stock-by-crop and status-distribution charts, an audit table, and a browser print action.

Shared application controls include dashboard navigation, notification display, a reference-date selector, add-crop action, threshold settings, and demo reset.

4. Decision rules

Crop age is computed as the number of calendar days between the harvest date and the selected reference date. Default classifications are:

Classification	Default age boundary	Recommendation
Stable	90 days or less	Continue Monitoring
Aging	91–270 days	Review Soon
Priority	More than 270 days	Review / Sell First

When humidity reaches or exceeds 75%, the application generates a storage-inspection advisory. Age and humidity classifications are recalculated as state changes.

These are prototype rules, not crop-specific validated storage standards. Although a high-temperature threshold exists in the settings data model, temperature does not currently drive a warning or recommendation.

5. Seeded demonstration scenario

The default simulation date is September 25, 2026. The initial data consists of:

Crop	Quantity	Harvest date	Location	Default age category
Rice	2,000 kg	July 10, 2026	Silo Bay A-1	Stable
Wheat	1,200 kg	January 15, 2026	Silo Bay B-2	Aging
Maize	800 kg	December 1, 2025	Granary Vault 3	Priority
Groundnut	500 kg	August 20, 2026	Dry Warehouse Bay 1	Stable

The seeded total is 4,500 kg, against a nominal capacity of 6,250 kg (72% utilization). Default simulated conditions are 28°C and 72% humidity. All listed figures are demonstration values.

6. Technical architecture

- Frontend: React 19 and TypeScript.
- Build and development: Vite.
- Styling: Tailwind CSS 4 and custom CSS.
- Charts: Recharts.
- Icons: Lucide React.
- State management: React component state and memoized derivations; no external state library.
- Data flow: `App.tsx` owns inventory, reference date, thresholds, and storage-condition state. It evaluates crop records and generates recommendations, then passes the resulting values and actions to page components.

Important project areas:

Area	Responsibility
`src/App.tsx`	Root state, page navigation, dialogs, and action handlers
`src/types/index.ts`	Shared TypeScript data types
`src/data/mockData.ts`	Initial crops, simulated readings, and default thresholds
`src/utils/calculator.ts`	Age calculation, status evaluation, and recommendation generation
`src/pages/`	Overview, inventory, storage, recommendation, and report screens
`src/components/`	Reusable dashboard components and dialogs

7. Data handling and integrations

The project has no server-side components or network API integration. Inventory, threshold, simulation-date, and environment changes exist only in the running browser session and are reset on page refresh. “Export CSV” creates a client-side file; “Print Report” invokes the browser print dialog.

No live IoT sensor data, user account, permissions, commodity prices, notifications service, or external agronomy dataset is connected. The pitch deck’s roadmap references possible future IoT, market-data, and predictive-optimization capabilities; these should be treated as future ideas rather than delivered features.

8. Observations and limitations

1. Persistence is absent. User changes are not saved across reloads or shared between users.

2. Environmental readings are simulated. Slider changes do not interact with real sensors.

3. Some capacity displays are static. The storage panel and sidebar show fixed 72% / 4,500-of-6,250 kg values, which may disagree with inventory after edits. Other areas calculate totals dynamically.

4. Temperature threshold is unused in decisions. Temperature is displayed and adjustable, but it does not currently cause an alert.

5. Threshold validation is incomplete. The UI constrains input ranges, but does not comprehensively validate relationships between thresholds before saving.

6. Storage bay mapping is fixed. The Storage page maps four known locations to four predetermined bays and falls back to seeded crop indexes.

7. Inventory search scope differs from its hint. Search covers crop name and storage location, not notes.

8. Some labels are illustrative. For example, the overview displays a fixed “+12% vs last month” badge and other demo status labels.

9. README is generic. The project README still describes the starter React/Vite template instead of this product.

10. Bundle size warning. The production JavaScript bundle is approximately 713 kB minified; Vite reports a chunk-size advisory.

11. Lint warnings remain. Oxlint reports a component declared during render in `InventoryAgingChart.tsx` and synchronous state updates in an effect in `AddEditCropModal.tsx`.

12. No automated test suite is configured. The package scripts include build and lint, but no test script.

9. Build and validation status

Validation run during report preparation:

- `npm run build` — passed, including TypeScript project build and Vite production build; emitted the large-chunk advisory.
- `npm run lint` — completed with two warnings listed above.

- No configured automated tests were found in `package.json`.

10. Recommended next steps

Before production use

- Replace demo state with a validated API and persistent database.
- Define authentication, roles, audit history, and data ownership.
- Validate crop, quantity, date, and threshold inputs at both client and server boundaries.
- Confirm crop-specific storage recommendations with qualified agronomic sources.
- Integrate sensor readings with timestamps, sensor identity, missing-data handling, and alerting policies.
- Make capacity, bay allocation, and all dashboard metrics derive from a single consistent data model.

Product and engineering improvements

- Update the README with setup instructions, feature overview, data model, and known limitations.
- Add unit tests for age boundaries, invalid/future dates, threshold changes, and recommendation generation.
- Add component or integration tests for crop CRUD, filters, exports, and key alert scenarios.
- Fix the two lint warnings and split or defer chart code if bundle performance warrants it.
- Add a real report-download format and ensure CSV escaping handles commas, quotes, and newlines in user data.
- Review accessibility, responsive layouts, and print styles before deployment.

11. Overall assessment

SmartGranary demonstrates a coherent concept: combine crop-lot tracking with transparent age-based prioritization and environmental inspection prompts. The interface and principal demo flows are implemented, and the production build succeeds. The present system should be described as an interactive MVP/prototype. Its sample rules and simulated data are not sufficient for operational decisions until persistence, real data integration, validated agronomic guidance, and automated testing are added.